

Structured query Language(MYSQL)



All of the contents are Taken from:

=>fundamentals of database system 7th-edition

=>W3schools.com

introduction.....	4
Core-concepts	4
MySQL-Fundamentals	6
Setting up Database.....	6
Default-Key	7
MYSQL-DataTypes.....	7
Keys & Constraints.....	11
Primary-Key	11
unique	11
AUTO INCREMENT	12
Basic Queries.....	13
INSERT-INTO	13
Select	14
SELECT-DISTINCT	14
UPDATE	15
DELETE	15
Condition-Statement	16
Logical-Statements.....	18
order-by	19
Not-statement	20
Null-Safety.....	20
Database Design.....	21
FOREIGN KEY	21
Database-Relation	23
Database Joins	26
introduction	26
INNER JOIN	26
LEFT JOIN	27
RIGHT JOIN	27
Cross JOIN	28

Aggregate Functions	29
Introduction	29
Min&Max.....	30
Count().....	30
With name alias:	31
DISTINCT	31
SUM().....	31
Database-Design.....	34
Table Manuplation	34
Advanced SQL	34
Group by	34
order by	35
Prodcedures.....	35
Views	36
Indexing.....	37
Subquires	37
Transaction.....	37

introduction

Core-concepts

what's data-models ?

- let's assume we're will open library ,
- we **need to draw a design** for **place** where **book will be stored**
- we need to know **what kind of books will be stored ?**
- is there any **relation** between books
- for example is **department** of **book-x** is same as **book-y** ?

what's Entites(Tables) ?

- **tables** contain **data** about **business** requirements like (**user** , **orders**)
- let's assume that we will have **resveration system** for reserving books online
- we will need **Table** that will contain **customer data** like **name** , **address**
- we will need **Table** that will contain **order data** like **order-name** , **order-price**
- any **column** exists within **entitiy** is called **Attribute || Field**
- any **row** exists within **entitiy** is called **Record**

what's Relationships ?

- it means how **records** is **linked togthter** with each other from **different table**
- let's assume we're **creating** a **management system** for our **library**
- we need **customer-table** connected to **products** , but what's the **relation between each other ?**
- **many customers** can buy **many products** , **many products** can be acquired to **many users** then relation between these two is **many : many**

what's relational databases ?

- ✓ it's a type of **database** that **store data** in **form of table**
- ✓ its **link** them **togthter by comman column**
- ✓ for Example => **Postgresql** , **mysql**

what's non-Relational Databases ?

- it's type of database it doesn't store data in table-form Column || Row
- it store data in form flexible data-model like documents , key-value pair
- it used in manage unstructured , semi-structured , structured data efficiently
- it used in handling Big-data efficiently
- assuming that we have 50 books each book stored in shelf contained in room
- when we need to store a new book we add a new room with a new shelf :)

what's Database management system ?

- software system used in **create , maintain , provide access to user databases**

what's Normalization ?

- process which **sorting data** in database to prevent:
 - **duplication**
 - **improve data-integrity**

SQL:

- Stands for **Structured query-language**

database object:

- **table is a collection of row & column**
- **Where column called field & row called record** like:

CustomerID	CustomerName	ContactName
1	Alfreds Futterkiste	Maria Anders
2	Ana Trujillo Emparedados y helados	Ana Trujillo
3	Antonio Moreno Taquería	Antonio Moreno

Statements:

- **it means SQL code**

Like this next statement:

- Select * from customer;
- **this used for printing all attributes** (Columns from customer-table)
- any database consists of more than **one table and each table identified with name & their records is linked together**

CustomerID	CustomerName	ContactName
1	Alfreds Futterkiste	Maria Anders
2	Ana Trujillo Emparedados y helados	Ana Trujillo
3	Antonio Moreno Taquería	Antonio Moreno

Notes

- This table contain three columns **with 3-records**
- SQL keyword **case insenstive which means select = SELECT**
- each SQL-statement must end with semi-colon (;)

MySQL-Fundamentals

Setting up Database

Creating Databases

syntax:

```
CREATE DATABASE NAME;
```

Example:

```
CREATE DATABASE Studnets;
```

existence Checking

syntax:

```
create database if not exist name
```

Deleting database

syntax:

```
DROP DATABASE NAME;
```

Example:

```
DROP DATABASE Studnets;
```

Creating Tables

syntax:

```
CREATE TABLE NAME( Column-1 datatype, Column-2 datatype)
```

Example:

```
DROP DATABASE Studnets;
```

existence Checking

syntax:

```
create table if not exist name
```

Deleting Table

syntax:

```
DROP TABLE NAME;
```

Example:

```
DROP Table Students;
```

Default-Key

What is DEFAULT?

- keyword in MySQL used to **assign a value automatically to a column** if **value isn't provided during insertion**.

Why to use DEFAULT?

- To avoid NULL values
- To provide fallback values

Example:

- If you have a column status, you may want it always to be "active" unless specified.

When to use DEFAULT?

- When a column usually has a common value
- When some fields are optional
- When you want automatic values like:

CURRENT_TIMESTAMP for dates , "guest" for usernames , 0 for counters

Syntax of DEFAULT

Creating a table with DEFAULT

```
CREATE TABLE table_name (  
    column_name data_type DEFAULT default_value  
);
```

Adding DEFAULT to existing column

```
ALTER TABLE table_name MODIFY column_name data_type DEFAULT default_value;
```

MYSQL-DataTypes

syntax:

```
CREATE TABLE NAME (  
COL-NAME DATATYPE ,  
);
```

Example:

```
CREATE TABLE users (  
  iD Char(25),  
);
```

String

CHAR(n)

- When to use: **Fixed-length text** that always has the same number of characters.
- Examples: **Country codes** ("US", "EG"), **fixed-length ID codes**.
- Important: **Only use for small, fixed-length fields. Using it for long variable text is wasteful.**
- n: **size of name which can't Exceed it**

VARCHAR(n)

- When to use: **Text of variable length**, most common for names, emails.
- Examples: **"Mohamed", "Ahmed"**, emails, addresses.
- Important: **Very common — basically every text field you store that isn't huge.**
- n: **size of name which can't Exceed it**

TEXT

- When to use: **Large text fields**, more than 255 characters.
- Examples: **Blog posts**, product descriptions, user comments. , chat-mesages
- Important: **Use only for long text; can't have default values in MySQL.**

Binary

BINARY(n)

- When to use: **Fixed-length binary data (non-text).**
- Why: **To store raw bytes** that always have the same size.
- Examples: **Password hashes**, encryption keys.
- Important: **Only for small, fixed-size binary values.**

VARBINARY(n)

- When to use: **Variable-length binary data**, like files or images.
- Why: **Saves space** compared to BINARY if size varies.
- Examples: **Small images**, file blobs, serialized objects.

- Important: **Good for files, but for really big files, use BLOB.**
- n: **size of name which can't Exceed it**

BLOB

- When to use: **Large binary objects.**
- Why: **Can store big data like images, videos, PDFs.**
- Examples: **User profile pictures, video uploads, documents.**
- Important: **Only for really large binary data — slower to query than VARCHAR/VARBINARY.**

Numeric

INT

- When to use: **Most common for whole numbers, IDs, counters.**
- Why: **Efficient storage and works well with auto-increment.**
- Examples: **user_id, order_id, product_count.**
- Important: **Default go-to integer type for almost all projects.**

BIGINT

- When to use: **Very large numbers that exceed INT range.**
- Why: **Handles huge numbers, e.g., global counters, big transaction IDs.**
- Examples: **Total views on a social media platform, large financial IDs.**
- Important: **Use only when INT is not enough.**

DECIMAL(p,s) / NUMERIC(p,s)

- When to use: **Exact decimal numbers, mostly money or financial data.**
- Why: **Avoids rounding errors, ensures accuracy.**
- Examples: **price, balance, tax_rate.**
- Important: **Always use for money; do not use FLOAT/DOUBLE for financial data.**

p = Precision

- **Total number of digits allowed in the number both before and after the decimal point.**

Example:

- **For DECIMAL(5,2)**
- **the precision is 5, so values can have up to 5 digits in total**

s = Scale

- **Number of digits** after the decimal point

Example:

- **For DECIMAL(5,2)**
- **the scale is 2**, so exactly up to 2 digits can appear after the decimal point like 123.45.

FLOAT / DOUBLE

- **When to use:** **Approximate decimals** for scientific or measurement data.
- **Why:** **Uses less storage** than DECIMAL but can have rounding errors.
- **Examples:** **percentages**, calculations where exact precision is not critical.
- **Important:** **Rarely used** for business apps; mostly for engineering or stats projects.

Date / Time

DATE

When to use: **When you only care about the date** (no time).

Why: **Simple and efficient** for calendar-based data.

Examples: **birth_date**, **exam_date**, **event_date**.

Important: **Stores only YYYY-MM-DD**. No hours or minutes.

DATETIME

- **When to use:** **When you need both date and time**, and accuracy matters.
- **Why:** **Most commonly used timestamp** in applications.
- **Examples:** **created_at**, **signup_date**, **order_time**.
- **Important:** **Does NOT auto-update** unless you explicitly set it.

TIMESTAMP

- **When to use:** **When you want the time to auto-update** on row changes.
- **Why:** **Very useful** for tracking updates automatically.
- **Examples:** **updated_at**, **last_login**.
- **Important:** **Can auto-update** with **CURRENT_TIMESTAMP**, but has a limited date range compared to DATETIME.

Keys & Constraints

Primary-Key

what's primary-key ?

- made for identifying each record uniquely
- based on last sentence it must acquire unique values
- each table can have only one primary-key
- it can be multiple field

live-example:

in university each student has ID , where this ID is the only for student not others

Syntax:

```
CREATE TABLE Name(  
COL-1 DATATYPE PRIMARY KEY ,  
COL-2 DATATYPE ,  
)
```

Example:

```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255),  
    PRIMARY KEY (ID,LastName)  
);
```

how can i indicate which table will contain primary-key ?

- just look at unique value

unique

what's unique keyword ?

- it means that data that can't be duplicated like national-id , photo

syntax:

```
CREATE TABLE Name(  
COL-1 DATATYPE PRIMARY KEY ,  
COL-2 DATATYPE unique )
```

Example:

```
CREATE TABLE users(  
id INT PRIMARY KEY AUTO_INCREMENT,  
name varchar(100),  
email varchar(100),  
nationl_number int unique ,  
image BLOB  
);
```

what's the difference between primary-key and unique ?

unique	primary-key
it can be more than one column	it must be one column
used in prevent conflict or duplication of data	it used for indicating records of db and manging relation between tables

AUTO INCREMENT

What is AUTO_INCREMENT?

- Used mostly with Primary Key
- MySQL adds the next number automatically
- You don't need to insert the value yourself

Why do we use it?

- To guarantee unique IDs
- To avoid manual ID handling
- To prevent duplicate primary keys

Example:

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20
4	name	name2	45

Syntax:

```
CREATE TABLE NAME(  
COL-1 DATATYPE PRIMARY KEY AUTOINCREMENT,  
COL-2 DATATYPE,  
COL-3 DATATYPE,  
)
```

Example:

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY AUTOINCREMENT,  
    username VARCHAR(100),  
    email VARCHAR(255)  
);
```

Important Notes

- **Works only with integer types** (INT, BIGINT, etc.)
- **Column** must be: **PRIMARY KEY** or **UNIQUE**
- Each **table** can have only **one AUTOINCREMENT column**
- **If a row is deleted, the number is not reused**
- You can start from a custom number

Basic Queries

INSERT-INTO

what's INSERT ?

- **means add data within database**

Inserting data methods

- We have two ways to write it in:

With columns-name

```
INSERT INTO Table-name (column-name, column-name) Values("V","V");
```

Without column-name but in correct orders

```
INSERT INTO Table-name Values("V","V","V")
```

Notes:

- you must insert into these columns based on their order

Example:

```
INSERT INTO Customers (x , y ,z) VALUES ('A','B','C');
```

Select

what's Select ?

- used in selecting data from database

Syntax:

SELECT column-1 , column-2 FROM table-name

Example:

Assume that we have the following table with name users and we need to select id,name only=> **SELECT id ,name FROM users**

id	name
1	X
2	Y
3	Z

what's select all ?

- select all data from database

What if we want to select all:

- SELECT * FROM table-name

On the last example:

- SELECT * FROM users

SELECT-DISTINCT

what's distinct means ?

- it means to select unique data

- why to use , Bec sometimes there's duplicates in column

- Which look like filtering the returning data from table

Syntax:

SELECT DISTINCT column-1, column-2 FROM table-name

Assume we have the following database with name called national:

City	PostalCode	Country
Berlin	12209	Mexico
México D.F.	05021	Mexico
México D.F.	05023	UK

SELECT DISTINCT country , city FROM national

UPDATE

what's update ?

- used in modifying records within table
- but here we're updating data based on condition to indicate which record will be updated

Syntax:

UPDATE Table-name Set Column = V, Set Column-n = V WHERE condition

Example:

Student-table

ID	Name	Age	Grade	City
200036827	Mohamed	20	20	Portsaid
200036828	Youssef	20	20	October
200036829	ahmed	19	19	sohag

UPDATE Student SET Name = 'yasser' , SET Age = 21 WHERE ID = 200036827;

ID	Name	Age	Grade	City
200036827	yasser	21	20	Portsaid
200036828	Youssef	20	20	October
200036829	Ahmed	19	19	sohag

updating more than one record

- we need them to have same condition like this

Example:

UPDATE Students SET City ="B" WHERE Grade = 20

ID	Name	Age	Grade	City
200036827	yasser	21	20	B
200036828	yossef	20	20	B
200036829	ahmed	19	19	C

DELETE

what's Delete ?

- at deleting in order to delete the desired data without any issue

- we need to use condition as update
- Delete-statement Deleting full Row

Syntax:

```
DELETE FROM Table-name WHERE Condition
```

Student-table:

ID	Name	Age	Grade	City
200036827	Mohamed	20	20	Portsaid
200036828	yossef	20	20	October
200036829	ahmed	19	19	sohag

```
DELETE FROM Table-name WHERE Condition
```

ID	Name	Age	Grade	City
200036828	yossef	20	20	October
200036829	ahmed	19	19	sohag

Note:

- To delete entire-data from table use

Syntax:

```
DELETE FROM Table-name
```

Example: =>DELETE FROM Students

ID	Name	Age	Grade	City

Condition-Statement

Where-statement

what's where statement ?

- statement that used in adding conditions

Syntax:

```
SELECT Column-Name FROM table-name WHERE condition
```

Let's assume that we have users-table with repeated data like this one:

ContactName	Address	City	PostalCode
Maria Anders	Obere Str. 57	Berlin	12209
Ana Trujillo	Avda. 2222	México D.F.	05021

Anton Moreno	Mataderos 2312	México D.F.	05023
Thomas Hardy	120 Hanover Sq.	London	WA1 1DP

SELECT ContactName **FROM** users **WHERE** City ="Mexico";

ContactName	Address	City	PostalCode
Ana Trujillo	Avda. 2222	México D.F.	05021
Anton Moreno	Mataderos 2312	México D.F.	05023

Notes:

- Used with all statements
- Numerical-value not quoted like this one:
- Where CustomerID = 1;

Operators for where-statement:

Where v is value :)

operator	Description	Syntax
=	Equal	WHERE col-n = v
>	Bigger-than	WHERE col-n > v
<	Less-than	WHERE col-n < v
>=	Bigger-than or equal	WHERE col >= v
<=	Less than or equal	WHERE col-n <= v
<>	Not equal main-operator	WHERE col-n <> v
!=	Not equal in other version	WHERE col-n != v
LIKE	To search for pattern	
BETWEEN	For a certain range	
IN	For multiple values	

Like-syntax:

WHERE col-n like 'letter%' => returning all records starts letter

For ex:

SELECT * FROM Customers WHERE City like 'S%';

Between-syntax:

WHERE col-n BETWEEN v AND v

For ex:

SELECT * FROM Products where ProductID between 1 and 5

In-syntax:

WHERE Col-n in ('V' , 'V') => v is not case-sensitive

For-ex:

```
SELECT * From Customers
```

```
WHERE Country in ("paris","London")
```

Logical-Statements

AND-Statement

- From programming and-operator is returning result if two conditions are true

- here in SQL is same

Syntax:

```
SELECT * FROM tab-N WHERE Condition AND Condition AND Condition .....
```

Syntax With OR:

```
SELECT * FROM tab-N WHERE AND (conditon OR condition)
```

Let's that we have this table with name Customers:

ContactName	Address	City	PostalCode	Country
Maria Anders	Obere Str. 57	Berlin	12209	Germany
Ana Trujillo	Avda. de la	México D.F.	05021	Mexico
Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden
Hanna Moos	Forsterstr. 57	Mannheim	68306	Germany
Frédérique	24, place Kléber	Strasbourg	67000	France

```
SELECT * FROM Customers WHERE Country = 'Mexico' AND City ='México D.F.' ;
```

ContactName	Address	City	PostalCode	Country
Ana Trujillo	Avda. de la 2222	México D.F.	05021	Mexico
Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Or-Statement

- as we know from programming or returning result if one condition at least is true

Syntax:

```
select * from table where condition or condition
```

Example:

```
SELECT * FROM Customers WHERE
```

```
CustomerName LIKE 'G%') OR (CustomerName LIKE 'R%'))
```

and-or statements

what's and-or ?

- it means using both and & or statements
- used when you're searching with two things one is 100%(and) sure while other not (or)

live-Example:

I'm searching on person it's called ali or maybe Ahmed but i'm sure that his age > 22

syntax:

select * from table where (condition or condition) and conditionZ

Example:

select * from x where

(name = "Sara Ahmed" OR name = "Youssef Samir") and age >22;

order-by

what's order by ?

- it means to make data organized based on specific column
- it's default Ascending and alphabetical
- you can set it manually by ASC / DESC

syntax:

SELECT col-n , col-n FROM table-name Order by Col-name

For different orders of columns :

SELECT * FROM tab-n ORDER BY col-n ASC , col-n DESC

For numerical-value:

Let's assume that we have table with name =>Products:

SupplierID	CategoryID	Price
23	3	16.25
9	7	30
25	5	9

SELECT * FROM Products

ORDER BY Price=>

SupplierID	CategoryID	Price
25	5	9

23	3	16.25
9	7	30

Notes:

- Default value will be asc => ascending s
- accept more than one-column with different sortings like:

SELECT * FROM Customers ORDER BY ContactName DESC , City ASC

CustomerName	ContactName	Address	City
Rancho grande	Sergio	Av. del	Buenos
Océano Atlántico Ltda.	Yvonne	Ing. Gustavo	Buenos Aires
Cactus Comidas para	Patricio	Cerrito 333	Buenos Aires
Piccolo und mehr	Georg	Geislweg 14	Salzburg
Ernst Handel	Roland	Kirchgasse 6	Graz
Suprêmes délices	Pascale	Boulevard Tirou,	Charleroi
Maison Dewey	Catherine	Rue Joseph-Bens	Bruxelles

Not-statement

what's Not-statement ?

- as we know it's a reverse operator that return opposite results

Syntax:

SELECT col-n FROM table-name WHERE NOT Condition;

Example:

SELECT * FROM Customers WHERE NOT Country='spain'

Notes:

- it can be used with any operators

For Example:

SELECT * FROM Customers WHERE CustomerID NOT BETWEEN 1 AND 5

Null-Safety

what's null ?

- it's a value that contain nothing empty i mean

what's Null Safety ?

- it means preventing any null value that any column can contain

why it's so important ?

- at any big project we're creating a very lage database

- if there's only one null value within can cause a very big Loss for project

Example on null value:

ID	Name	year
200036827	Ahmed	2000
200036828	Mohamed	
200036829	karim	2002

Syntax for checking if any column has null-value by two ways:

how can we check for null value ?

- just by following syntax:

SELECT Column-names FROM table WHERE Column-name IS NULL

Example:

SELECT * FROM Orders WHERE OrderID IS NULL :

Number of Records: 0				
OrderID	CustomerID	EmployeeID	OrderDate	ShipperID

how can we check for value ?

- just by following syntax:

SELECT Column-names FROM Table-name WHERE Column-name IS NOT NULL

Example:

SELECT * FROM Orders WHERE OrderID IS NOT NULL;

Number of Records: 196				
OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10248	90	5	7/4/1996	3
10249	81	6	7/5/1996	1
10250	34	4	7/8/1996	2

Used in

- Creating Field of Database like id INT PRIMARY KEY AUTOINCREMENT NOT NULL

Database Design

FOREIGN KEY

what's FOREIGN-key ?

- used in linking table between each others

Notes:

- table with primary key is the father
- table with foregin key is the childs

why Linking ?

- in order to acheving relationships between tables

Live-Example:

let's assume we're working on a learning platform , data that we have:

- users

- courses

- enrollments

here we need to determine:

- which course is enrolled

- which user enrolled which course

here's Enrollments database:

```
CREATE TABLE Enrollments (  
  enrollment_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  course_id INT,  
  enroll_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) Users(user_id),  
  FOREIGN KEY (course_id) Courses(course_id)  
);
```

how can i indicate which table will contain foregin-key ?

- make your tables normally like last example users , courses , enrolements

- ask your self which table will be the linker between two of them

syntax:

```
CREATE TABLE A(  
  COL-X DATATYPE PRIMARY KEY ,  
  COL-Y DATATYPE ,  
)  
CREATE TABLE B (  
  COL-J DATATYPE PRIMARY KEY ,  
  COL-2 DATATYPE ,  
)  
CREATE TABLE C (  
  COL-X DATATYPE PRIMARY KEY ,  
  COL-J DATATYPE ,
```

```
FOREIGN KEY(COL-X) REFERENCES TABLE-A(COL-X)
FOREIGN KEY(COL-J) REFERENCES TABLE-B(COL-B))
```

Example:

```
CREATE TABLE if not exists users(
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  name varchar(100),
  email varchar(100),
  nationl_number int unique );

CREATE TABLE if not exists products(
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  title varchar(60),
  txt text
);

CREATE TABLE if not exists orders(
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT ,
  product_id INT ,
  foreign key (product_id) REFERENCES products(product_id) ,
  foreign key(user_id) REFERENCES users(user_id)
);
```

Database-Relation

what's database relation ?

- it define how data is Linked across the tables

why to use database relation ?

- to link Entities together
- to prevent data-conflict between each other
- to acheiving data integrity(سلامة البيانات من الضياع او التداخل)

Classisification of relations

- one to one

- one to many
- many to many
- many to one
- self relationship

one to one

- it means each record in table-1 linked with one record in table-2 , vice-versa

Example:

- each user has one profile

one to many

- it means each record in table-1 linked with multiple records in table-2 , vice-versa

Example:

- each user is enrolled in multiple courses

many to many

- it each multiple records in table-1 is linked with multiple records in table-2 , vice-versa

Example:

- many users reserved many orders

many to one

- it means multiple record from table-1 is linked with one record with table-2 , vice-versa

Example:

- many students is enrolled with specific courses

self relationship

- it means that record of table-1 is linked together

Example:

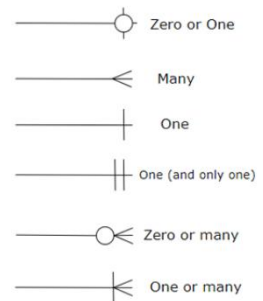
- manager has team to manage where team is users , manager is user

Symbols for diagram

- until now we need to know this symbols only
- later on it will be used in E.R.D Diagram

how can we indicate our relation between Entities ?

- just by looking at business logic
- we design database based on requirements of our project



Simple-task

A company has:

- Employees
- Departments
- Projects

Business Rule || logic:

- Each employee works in exactly one department.
- Each department can have many employees.
- Each project can have many employees working on it.
- Each employee can work on many projects.

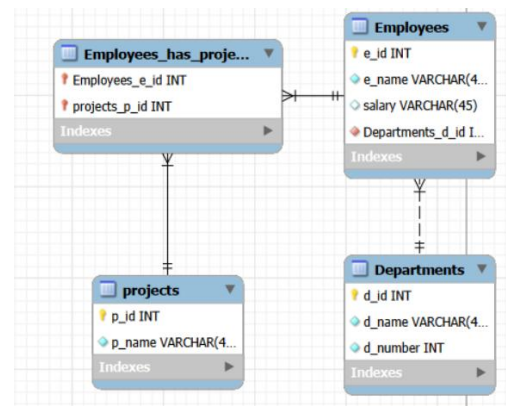
your task is:

- Identify the relationship types
- Draw an E²RD

sol

where can we reach Business Rule || logic?

- from **Client** if it for **freelance**
- from **project manager** if it for **company**
- from **google** or **you create it** if it for **portfolio**
- From Design



Database Joins

introduction

what's joins ?

- we use it in **Merging data from different tables together**

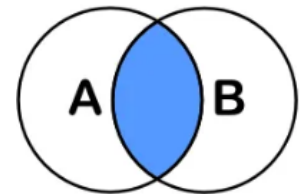
General Syntax:

```
select A.Column , B.column from A {join-type} join B on A.col = A.Col;
```

INNER JOIN

what's inner join ?

- way to **present matched data from two tables or more**
- this diagram **shows both table where the interconnected part is the matched data**



syntax:

```
select T1.Column , T2.column from T1 inner join T2 on T1.col = T2.Col;
```

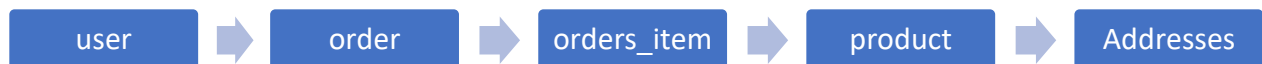
Example:

```
select users.first_name , users.last_name , orders.total_price , users.phone_number  
from users inner join orders on users.id = orders.user_id;
```

how can order linked table together ?

based on relation look at this example:

we need to display order which is username | product-title | order-price | user_address
then we will link more than three table bec product to relation m:m



and here's the query:

```
SELECT users.first_name, orders.total_price, products.title, order_items.quantity,  
addresses.street_number addresses.apartement_number FROM users  
INNER JOIN orders ON users.id = orders.user_id INNER JOIN order_items ON  
orders.id = order_items.order_id INNER JOIN products ON order_items.product_id=  
products.id INNER JOIN addresses on users.id = addresses.user_id
```

Selecting based on Condition

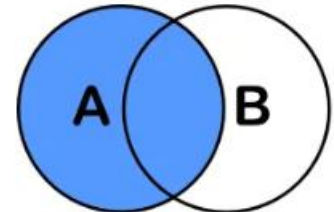
- just determine based on what column to apply condition then make it :)

```
select users.first_name , users.last_name , orders.total_price users.phone_number ,
addresses.street_number, addresses.apartement_number
from users inner join orders on users.id = orders.user_id where users.id = 8;
```

LEFT JOIN

what's left join ?

- **presenting all of table A with matched part only of table B**
- this diagram **shows two tables where the all of Table A appear with the matched part only of Table B**



syntax:

```
select A.Column , B.column from A Left join B on A.col = B.Col;
```

Example:

```
SELECT employees.id, employees.name, employees.department_id,
departments.name from employees left join departments
on employees.department_id = departments.id
```

id	name	department_id	name
1	Mohamed	1	IT
2	Ahmed	2	HR
3	Ali	3	Marketing
4	Omar	NULL	NULL

Live-Example:

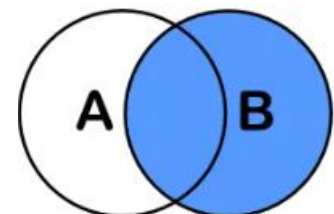
- we need to display product whether it's review or not and for sure
- product table - product reviews - catogeries table

```
select products.product_image, products.title, products.old_price,
products.stock, categories.name, product_reviews.rating FROM products INNER
join categories on products.category_id = categories.id left join product_reviews
on products.id = product_reviews.product_id
```

RIGHT JOIN

what's right join ?

- **presenting all of table B with matched part only of table A**
- this diagram **shows two tables where of all Table B appear with the matched part of Table A**



syntax:

```
select A.Column , B.column from A Left join B on A.col = B.Col;
```

Example:

```
SELECT employees.id, employees.name, employees.department_id,  
departments.name from employees Right join departments  
on employees.department_id = departments.id
```

id	name	department_id	name
1	Mohamed	1	IT
2	Ahmed	2	HR
3	Ali	3	Marketing
4	Omar	NULL	NULL
NULL	NULL	NULL	Finance
NULL	NULL	NULL	Sales

Live-Example:

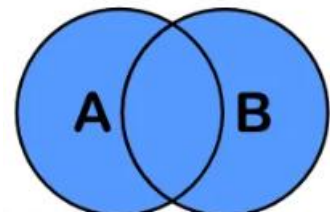
- from admin view we have product table catogeries table
- we need to display catogery to see what's that catogery need supplies

```
select products.product_image, products.title, products.old_price,  
products.stock, categories.name, FROM products Right join categories on  
products.category_id = categories.id
```

Cross JOIN

what's Cross join ?

- **presenting all of table A with all of table B**
- this diagram **shows two tables where the all Table A appear with all Table b**



syntax:

```
select A.Column , B.column from A Cross join B
```

Example:

```
SELECT employees.id, employees.name, employees.department_id,  
departments.name from employees cross join departments
```

id	name	department_id	name
1	Mohamed	1	IT
1	Mohamed	1	HR
1	Mohamed	1	Marketing

1	Mohamed	1	Finance
1	Mohamed	1	Sales
2	Ahmed	2	IT
2	Ahmed	2	HR
2	Ahmed	2	Marketing
2	Ahmed	2	Finance
2	Ahmed	2	Sales
3	Ali	3	IT
3	Ali	3	HR
3	Ali	3	Marketing
3	Ali	3	Finance
3	Ali	3	Sales
4	Omar	NULL	IT
4	Omar	NULL	HR
4	Omar	NULL	Marketing
4	Omar	NULL	Finance
4	Omar	NULL	Sales

Live-Example:

- from admin view we have product table catogeries table
- we need to display catogery to see what's that catogery need supplies

select **products.product_image**, **products.title**, **products.old_price**,
products.stock, **categories.name**, FROM **products** Right join **categories** on
products.category_id = categories.id

Aggregate Functions

Introduction

- it Return a Single Value of Column

General Syntax:

SELECT aggregate_function() from table name where condition

Aggregate Functions	Descriptiion
MIN	Return smallest value within selected column
Max	Return largest value within selected column
count	Return number of row numircal row
AVG	Return average value of numircal-column

Note:

- All Aggregate Functions ignore null values execept for Count-function

Min&Max

Usage:

From it's name it returns the smallest value from selected-column

Syntax:

Min:

SELECT MIN(Col-Name) FROM TableName where Condition

For Example:

ProductID	Price
1	10
2	11
3	12
4	13
5	14
6	15
7	16

SELECT MIN (Price) FROM Products;

Expected-name
10

And to remove this sentence we will need to know about alias

Syntax:

SELECT MIN(Column-name) AS (name as you want) like:

For Example:

SELECT MIN(Column-name) AS SmallestPrice from Products

SmallestPrice
10

الماكس بالظبط زيها ف مش محتاج اشرحها تاني

Count()

Usage:

From it's name used to count all records within row

Syntax:

SELECT COUNT (Column-name) FROM Table-Name WHERE Condition

For Example:

SELECT COUNT (Price) FROM Products;

ProductID	ProductName	SupplierID	CategoryID	Price
1	Chais	1	1	18
2	Chang	1	1	19
3	Aniseed Syrup	1	2	10
4	Chef Anton's	2	2	22
5	Chef Anton's	2	2	21.35
6	Boysenberry Spread	3	2	25
7	Uncle Bob	3	7	30
8	Cranberry Sauce	3	2	40

Output:

Expr1000
77

With name alias:

Syntax:

SELECT (Column-Name) as Total

Example:

SELECT COUNT (Price) as Total FROM Products;

Total
77

And to prevent duplicates we will use:

DISTINCT

Syntax:

SELECT (DISTINCT column-name) FROM Products;

For-Example:

SELECT COUNT(DISTINCT Price) FROM Products;

SUM()

Usage:

Finding total value of column with numbered records

Syntax:

SELECT SUM (Col-name) FROM Table-Name WHERE Condition

For Example:

SELECT SUM (Price) FROM Products WHERE ProductsID = 11

هنا هيشوف كل عمود و هيروح ل يشوف البرايس العنده و يجمعهم على بعض ف المفروض الناتج يبقى:

Expected-name
12743

Syntax:

SELECT SUM(Col-Name) as (name as you want) FROM Table-Name WHERE Condition

For-example:

SELECT OrderID, SUM(Quantity) AS [Total Quantity] FROM OrderDetails GROUP BY OrderID;

And it can be used with expression:

SELECT SUM(Quantity * 10) FROM OrderDetails;

Like:

Used with where for searching (conditions) and there's two operators used with it which them:

Percent-sign.(%):

Represent zero or one or multiple characters

Underscore-sign.(_):

Represents one or single character

Syntax:

SELECT Column-1 , Column-2 FROM Table_name WHERE Column-3 LIKE Pattern

Pattern:

حاجه زي الريجكس ف الجي اس طيب يعني ايه انا مثلا عاوز ارجع الدتا الفى مدينه لندن:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
11	B's Beverages	Victoria Ashworth	Fauntleroy Circus	London	EC2 5NT	UK
16	Consolidated Holdings	Elizabeth Brown	Berkeley Gardens 12 Brewery	London	WX1 6LT	UK
19	Eastern Connection	Ann Devon	35 King George	London	WX3 6FW	UK
53	North/South	Simon Crowther	South House 300 Queensbridge	London	SW7 1RZ	UK
72	Seven Seas Imports	Hari Kumar	90 Wadhurst Rd	London	OY15 4NR	UK

London

=>

Select * from Customers where City Like "L__nd__"

And for berlin:

Select * from Customers where City Like "B__li_ "

طبيب ايه دا انت كانك بتقوله بص انا عاوزك ترجعلي

اسم مدينه بيبدء ب حرف البي و بعد البي في حرفين و بعد الحرفين في ال

Li

و بعدهم في حرف

طبيب انا لو عاوز ارجع اي اسم مدينه فيها حرف ال بي مثلا يبقى هنا نستعمل البرسنت

Select * From Customers where City like "%B%"

و خد بالك هيرجعلك الكبيتل و الاسمول عادي جدا

طبيب انت هنا قولته انك عاوز اسم مدينه فيه حرف البي و ممكن يكون قبله او بعده حروف

Starts with:

طبيب انا لو عاوزه بيبدء بالحرف دا يبقى:

Select * from Table_name where col-1 like "letter%"

Ends With:

طبيب انا لو عاوزه يختار اي حاجه اخرها حرف

Select * from Table_name where col-1 like "%letter"

Starts & Ends With:

بيبدء بحرف معين و يخلص بحرف معين

Select * from Table_name where col-1 like "letter%letter"

Contains:

بيحتوى على كلمه او حاجه معينه:

Select * from Table_name where col-1 like "%letter%"

Without wildcard:

من غير يعني اي حاجه من الانا اخدتها

Select * from Table_name where col-1 like "word"

Database-Design

https://www.youtube.com/watch?v=VQGiO38c_EM

Table Manipulation

Describe table

Truncate Table

Modifying table

Advanced SQL

Group by

what's group by ?

- it mean grouping data based on specific Column
- usually used with aggregate functions

syntax:

```
SELECT COUNT(Column-1) , Column -2 FROM Table GROUP BY Column-2
```

Example:

```
SELECT customer, COUNT(*) AS orders_count FROM orders GROUP BY
```

Customer	orders_count
Mohamed	3
Ahmed	3
Ali	3
Omar	2
Youssef	3
Hassan	2
Karim	2

Live-example:

- in admin view we need to show in each catogery how many product

```
select categories.name, count(products.id) as product_count from products
```

inner JOIN **categories** on **products.category_id** = **categories.id** GROUP BY **categories.name**;

name	product_count
Electronics	8
Laptops	8
Smartphones	7
Accessories	9
Gaming	8
Home Appliances	9
Fashion	16

order by

Procedures

What's a Procedure ?

- like a function in Programming Languages

What problem does it solve?

- The main problem is reusing database operations.
- Imagine **you repeatedly need to create a user**.

Syntax:

```
DELIMITER //  
  
CREATE PROCEDURE procedure_name(  
    IN parameter_name DATA_TYPE ||  
)  
  
BEGIN  
    -- SQL statements  
  
END;  
  
DELIMITER //  
  
call procedure_name()
```

Notes:

- Procedures can be **parameterized** or **non-parameterized**

what's Delimiter ?

- character or sequence of characters that tells MySQL statement is finished

So why do we need DELIMITER with Procedures?

- Because a Procedure can contain multiple SQL statements.

Live-example With Procedure:

```
DELIMITER //

-- DROP PROCEDURE IF EXISTS create_user;

CREATE PROCEDURE create_user (
    IN p_first_name VARCHAR(50),
    IN p_last_name VARCHAR(50),
    IN p_email VARCHAR(255),
    IN p_password VARCHAR(250),
)
BEGIN INSERT INTO users (first_name,last_name,email,password)
VALUES (p_first_name,p_last_name,p_email,p_password);
END //

DELIMITER ;

CALL create_user('Mohamed','Yasser','test@example.com','123456','01234',NULL);
```

Deleting Procedure

```
DROP PROCEDURE IF EXISTS procedure_name
```

Views

What's a View?

- a virtual table based on a SELECT query.

What problem does it solve?

- A View mainly solves Repeating complex SELECT queries

syntax:

```
CREATE VIEW view_name AS SELECT column1, column2 FROM table_name
WHERE condition;
```

Live-Example on Views:

```
CREATE VIEW user_profile AS  
SELECT id , first_name, last_name, email, bio, telephone_number FROM users;
```

Intuition:

View is like taking a sub Table of Table

Indexing

Subquires

Transaction
